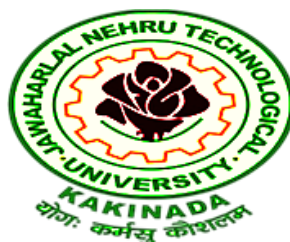


A Project Report

On

***AIR QUALITY PREDICTION BASED ON MACHINE
LEARNING***



A project report Submitted in partial fulfilment of requirements for the
Award of the degree of

**MASTER OF COMPUTER APPLICATIONS
SUBMITTED BY**

SWAPNA UGGAM

(23P31F0002)

Under the Esteemed Guidance of
Dr.M.VAMSIKRISHNA,M.Tech., Ph.D
HOD OF MCA



**ADITYA COLLEGE OF ENGINEERING
& TECHNOLOGY**

An Autonomous Institution

**(Approved by AICTE, Affiliated to JNTUK & Accredited by NBA, NAAC
with 'A++' Grade) Recognized by UGC under the sections 2(f) and 12(B)
of the UGC act 1956 SURAMPALEM
533 437, E.G.Dt, ANDHRA PRADESH.
2023-2025**

ADITYA COLLEGE OF ENGINEERING & TECHNOLOGY

An Autonomous Institution

(Approved by AICTE, Affiliated to JNTUK & Accredited by NBA, NAAC with 'A++' Grade)

Recognized by UGC under the sections 2(f) and 12(B) of the UGC act 1956

SURAMPALEM- 533 437, E.G.Dt, ANDHRA PRADESH

2023 - 2025

DEPARTMENT OF MCA



CERTIFICATE

This is to certify that the project work entitled, "**AIR QUALITY PREDICTION BASED ON MACHINE LEARNING**", is a bonafide work carried out by **SWAPNA UGGAM** bearing Regd.No:**23P31F0002** submitted to the requirements for the award of the Computer Applications in partial fulfilment of the requirements for the award of degree of **MASTER OF COMPUTER APPLICATIONS** from Aditya College of Engineering & Technology, Surampalem during the academic year **2024-2025**.

Project Guide

Dr.M.Vamsikrishna,M.Tech.,Ph.D

Head of the Department of DCA

Aditya College of Eng & Tech,

Surampalem-533437

Head of the Department

Dr.M.Vamsikrishna,M.Tech.,Ph.D

Head of the Department of MCA,

Aditya College of Eng& Tech,

Surampalem-533437.

EXTERNAL EXAMINER

DECLARATION

I hereby declare that the project entitled “**AIR QUALITY PREDICTION
BASED ON MACHINE LEARNING**”, to **Aditya College of Engineering &
Technology, Surampalem** has been carried out by me alone under the guidance of
Dr.M.Vamsikrishna.

Place: Surampalem

Date:

SWAPNA UGGAM

(23P31F0002)

ACKNOWLEDGEMENT

The satisfaction and euphoria that accompany the successful completion of any task would be incomplete without the mention of people who made it possible, whose constant guidance and encouragement crowned our efforts with success. It is a pleasant aspect that I have now the opportunity to express my gratitude for all of them.

The first person I would like to thank is my guide **Dr.M.Vamsikrishna, M.Tech.,Ph.D Head of the Department of MCA, Aditya College of Engineering & Technology, Surampalem**. His wide knowledge and logical way of thinking have made a deep impression on me. His understanding, encouragement and personal guidance have provided the basis for this project. He is a source of inspiration for innovative ideas and his kind support is well known to all his students and colleagues.

I wish to thank **Dr. M .Vamsikrishna Head of the Department, Aditya College of Engineering & Technology, Surampalem**, who has extended her support for the success of this project.

I also wish to thank **Dr. M. Sreenivasa Reddy, M.Tech, Ph.D, Principal, Aditya Engineering College, Surampalem**, who has extended his support for the success of this project.

I would like to thank all the **Management & Technical Supporting Staff** for their timely help throughout the project.

ABSTRACT

ABSTRACT

One major basic right is clean air that is integral to the concept of citizenship and it's while not a doubt, the responsibility of every subject to try to do his/her half to stay the air clean. Air quality prognostication has been looked into because the key answer of early warning and management of pollution. During this paper, we tend to propose an Associate in nursing air quality prediction system supported by a machine learning framework known as the sunshine GBM model, to predict air quality. This model, trained victimization lightweight GBM classifier, take meteorology knowledge jointly of sources for predicting the air quality thereby increasing the prediction accuracy by creating full use of obtainable abstraction data. the prevailing air quality observance stations and satellite meteorologic knowledge offer period air quality observance info that is employed to predict the trend of air pollutants within the future. The proposed system achieves a prediction accuracy of 92%, demonstrating its effectiveness in air quality prediction. With advancements in computational power and the availability of real-time environmental data, systems like ours pave the way for more informed decision-making, contributing to a cleaner and healthier future.

TABLE OF CONTENTS

CHAPTER NO	CONTENT	PAGE NO
	AKNOWLEDGMENT	4
	ABSTRACT	5
Chapter1.	Introduction	8
Chapter2.	Literature Survey	10
Chapter3.	System Analysis	12
3.1	Existing System	14
3.2	Proposed System	14
Chapter4.	System Design	15
4.1	System Architecture	16
4.2	UML Representation	17
Chapter5.	System Implementation	23
5.1	Modules	24
5.2	Requirements Specification	25
Chapter6.	Technology Description	26
6.1	Introduction to Python	27
Chapter7.	Sample code	35
Chapter8.	Testing	42
8.1	Types Testing	43
8.2	Test Cases	44
Chapter9.	Screen Shots	46
Chapter10.	Conclusion and Future work	56
10.1	Bibliography	58

INTRODUCTION

1. Introduction

Introduction

Air pollution poses a significant global threat, particularly in densely populated and industrialized urban areas. With rapid economic and societal growth—especially in developing nations like India and China—the concentration of harmful air pollutants at ground level has increased sharply. These pollutants, including nitrogen oxides (NO), carbon monoxide (CO), sulfur dioxide (SO₂), and particulate matter (PM_{2.5}), are known to cause severe health complications such as respiratory and cardiovascular issues. Both short- and long-term exposure to such pollutants can adversely affect human health, especially among vulnerable populations. To mitigate these effects, the development of an early warning system capable of accurate pollution forecasting is crucial. Such systems not only predict pollutant levels but also help issue timely health alerts to the public. Air pollution, both from ambient (outdoor) and household sources, is estimated to cause approximately 7 million premature deaths globally each year, underlining the urgent need for effective prediction and intervention tools.

Problem Statement & Objectives

This research focuses on building a predictive model to determine future air pollution levels using large datasets and machine learning techniques. The core objective is to develop an intelligent, data-driven application that leverages algorithms such as Convolutional Neural Networks (CNNs), Artificial Neural Networks (ANNs), and other supervised learning methods to deliver highly accurate Air Quality Index (AQI) forecasts. Traditional methods like statistical modeling and probability-based predictions were once used for AQI forecasting. However, they often fall short in terms of efficiency and adaptability. With the advancement of modern technology, real-time data can now be captured through sensors, enabling more precise and dynamic analysis of pollutant levels.

Significance of the Study

Accurate forecasting of pollutant concentrations enables early interventions, improves public health responses, and supports environmental policymaking. Machine learning, a branch of artificial intelligence, offers powerful tools for recognizing complex patterns in data. By employing supervised learning algorithms, this project aims to predict air quality with greater precision, making it easier to take timely actions and reduce health risks.

LITERATURE SURVEY

2. LITERATURE SURVEY

Literature Review

- Title:** *Air Pollutant Severity Reduction Using Bi-Directional LSTM Network*
Authors: Verma, Ishan; Ahuja, Rahul; Meisheri, Hardik; Dey, Lipika
Summary:
This study presents a predictive model for air pollutant severity using Bi-Directional Long Short-Term Memory (BiLSTM) networks. By combining three BiLSTMs that analyze short-term, long-term, and immediate pollution trends, particularly of PM_{2.5}, the model achieves high accuracy. Weather data integration further boosts performance. Experimental validation in New Delhi confirms the model's effectiveness over traditional techniques.
- Title:** *Urban Air Quality Based on Bayesian Network*
Authors: Yang, Raijin; Yan, Feng; Zhao, Nan
Summary:
This paper proposes the use of a Bayesian Network to evaluate urban air quality through a directed acyclic graph model. Using air quality data from Shanghai, the model is trained and validated, achieving a high prediction accuracy of 99.31%. The study demonstrates the practicality and precision of Bayesian networks in air quality assessment.
- Title:** *Evolving Differential Evolution Method with Random Forest for Prediction of Air Pollution*
Author: Kumar, Dinesh
Summary:
The paper introduces a hybrid model combining Differential Evolution (DE) strategies with Random Forest (RF) algorithms for forecasting air pollution levels. Using pollution data from Delhi and Patna, the approach shows improved performance over traditional classifiers. The model targets pollutants like PM_{2.5}, PM₁₀, CO, SO₂, and others, offering enhanced predictive reliability through ensemble learning.
- Title:** *Exploring the Utility of the Random Forest Method for Forecasting Ozone Pollution in Sydney*
Authors: Jiang, Ningbo; Riley, Matthew L.
Summary:
This research assesses Random Forest (RF) algorithms for ozone level prediction in Sydney, comparing its performance to traditional CART models. RF outperforms in key polluted regions, demonstrating higher forecasting precision. The study validates RF as a robust ensemble method for air quality forecasting, especially in complex urban environments.
- Title:** *Random Forest: A Classification and Regression Tool for Compound Classification and QSAR Modelling*
Authors: Svetnik, Vladimir; et al.
Summary:
This work examines Random Forest as a reliable tool for classification and regression, particularly in cheminformatics. The model utilizes bootstrap aggregation and random feature selection for tree construction. Tested on six datasets, RF offers high prediction accuracy, internal validation features, and effective variable importance ranking, making it suitable for diverse predictive applications.

SYSTEM ANALYSIS

3. SYSTEM ANALYSIS

3.1 EXISTING SYSTEM

The existing system aims to predict air quality using machine learning techniques. It utilizes historical air pollutant data collected from sensors and preprocesses the data by cleaning and normalizing it. The dataset is divided into training and testing sets, and feature selection is applied to identify relevant attributes. Various supervised machine learning algorithms, including Linear Regression, Support Vector Machine (SVM), Decision Tree, and Random Forest, are employed to predict air quality levels. The system's architecture involves data extraction, preprocessing, feature selection, training and testing using machine learning algorithms, and prediction. The goal is to develop a model that accurately predicts air quality levels for improved pollution management and public health protection.

DISADVANTAGES OF EXISTING SYSTEM:

The accuracy of air quality predictions heavily relies on the quality and coverage of sensor data.

If the sensor network is sparse or data quality is poor, it can affect the reliability of the predictions.

Algorithm: SVM, linear regression

3.2 PROPOSED SYSTEM

The proposed system aims to enhance air quality prediction by incorporating advanced machine learning algorithms like Random forest, decision tree, SVM. It includes ensemble learning, time-series analysis, and real-time monitoring for adapting to changing data. The system integrates external factors, improves model interpretability, and features a user-friendly interface.

ADVANTAGES OF PROPOSED SYSTEM:

By incorporating advanced machine learning algorithms and ensemble techniques, the proposed system can provide more accurate predictions of air quality levels, helping authorities and individuals make informed decisions.

Algorithm: Support Vector Machine (SVM), Decision Tree (DT), Random Forest (RF), Linear Regression.

SYSTEM DESIGN

4 . SYSTEM DESIGN

1.1 INTRODUCTION

System design is the process of planning and organizing different components of a software application to ensure smooth functionality, efficiency, and scalability. It defines how various parts of the system interact, how data flows between them, and how users will experience the application. A well-structured system design ensures that the software meets user requirements, performs efficiently, and is easy to maintain and upgrade.

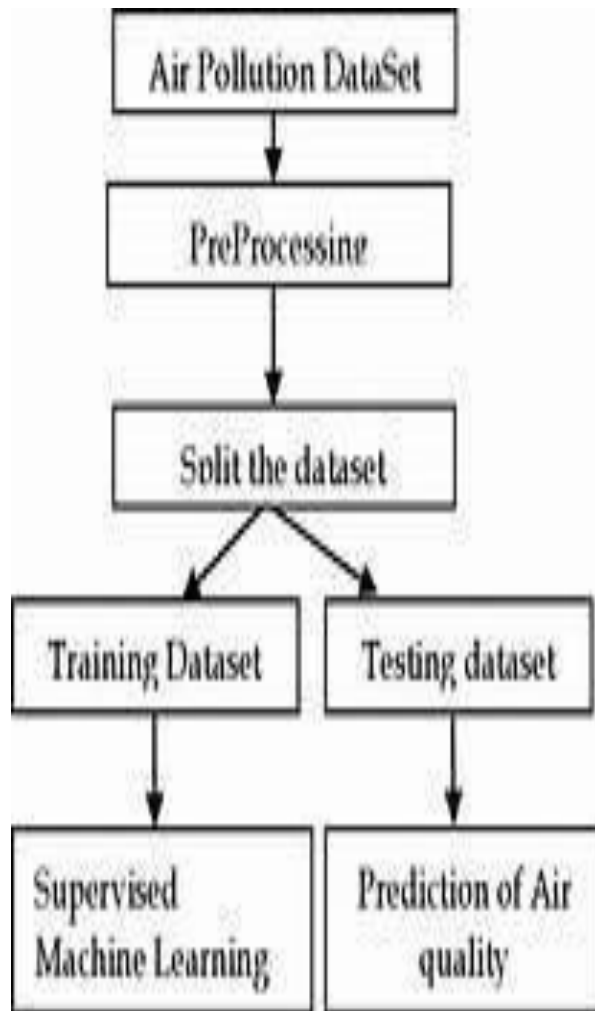
The system design of the Travel Plan project follows a structured and modular approach for better efficiency and maintainability. The frontend is built using Flutter, providing a smooth and interactive user experience. The backend is developed in Python, handling business logic and database operations. SQLite is used as the database to store user details, travel itineraries, and booking information. The application follows the MVC (Model-View- Controller) architecture, ensuring a clear separation between UI, logic, and data management. Data is retrieved and updated through RESTful APIs, allowing seamless communication between the Flutter frontend and the Python backend. The app also supports local storage (Shared Preferences) for caching frequently accessed data, improving performance and reducing dependency on network calls.

System design can be divided into two major parts:

- **High-Level Design (HLD)** – This focuses on the overall system architecture, including the database, servers, APIs, and interactions between major components. It defines how data will be stored, retrieved, and processed. For example, in a travel planning app, the database must efficiently store travel details, user information, and more while ensuring security and quick access.
- **Low-Level Design (LLD)** – This involves the detailed design of specific features, such as user login authentication, search filters, and payment gateways. It defines how each module will work internally, ensuring smooth operations and integration with other parts of the system. For instance, when a user searches for a trip, LLD defines how the system checks availability, updates the database, and sends a confirmation notification.

A good system design also considers security, reliability, and scalability. Security measures like data encryption, authentication protocols, and role-based access control help protect sensitive user information.

4.1 SYSTEM ARCHITECTURE



4.1.1 System Architecture

4.2 UML REPRESENTATION

UML stands for Unified Modeling Language. UML is a standardized general-purpose modeling language in the field of object-oriented software engineering. The standard is managed, and was created by, the Object Management Group. The goal is for UML to become a common language for creating models of object oriented computer software. In its current form UML is comprised of two major components: a Meta-model and a notation. In the future, some form of method or process may also be added to; or associated with, UML. The Unified Modeling Language is a standard language for specifying, Visualization, Constructing and documenting the artifacts of software system, as well as for business modeling and other non software systems.

The UML represents a collection of best engineering practices that have proven successful in the modeling of large and complex systems. The UML is a very important part of developing objects oriented software and the software development process. The UML uses mostly graphical notations to express the design of software projects

GOALS:

The Primary goals in the design of the UML are as follows:

1. Provide users a ready-to-use, expressive visual modeling Language so that they can develop and exchange meaningful models.
2. Provide extendibility and specialization mechanisms to extend the core concepts.
3. Be independent of particular programming languages and development process.
4. Provide a formal basis for understanding the modeling language.
5. Encourage the growth of OO tools market.
6. Support higher level development concepts such as collaborations, frameworks, patterns and components.
7. Integrate best practices.
- 8.

4.3 UseCase Diagram

A use case diagram in the Unified Modeling Language (UML) is a type of behavioral diagram defined by and created from a Use-case analysis. Its purpose is to present a graphical overview of the functionality provided by a system in terms of actors, their goals (represented as use cases), and any dependencies between those use cases. The main purpose of a use case diagram is to show what system functions are performed for which actor. Roles of the actors in the system can be depicted.

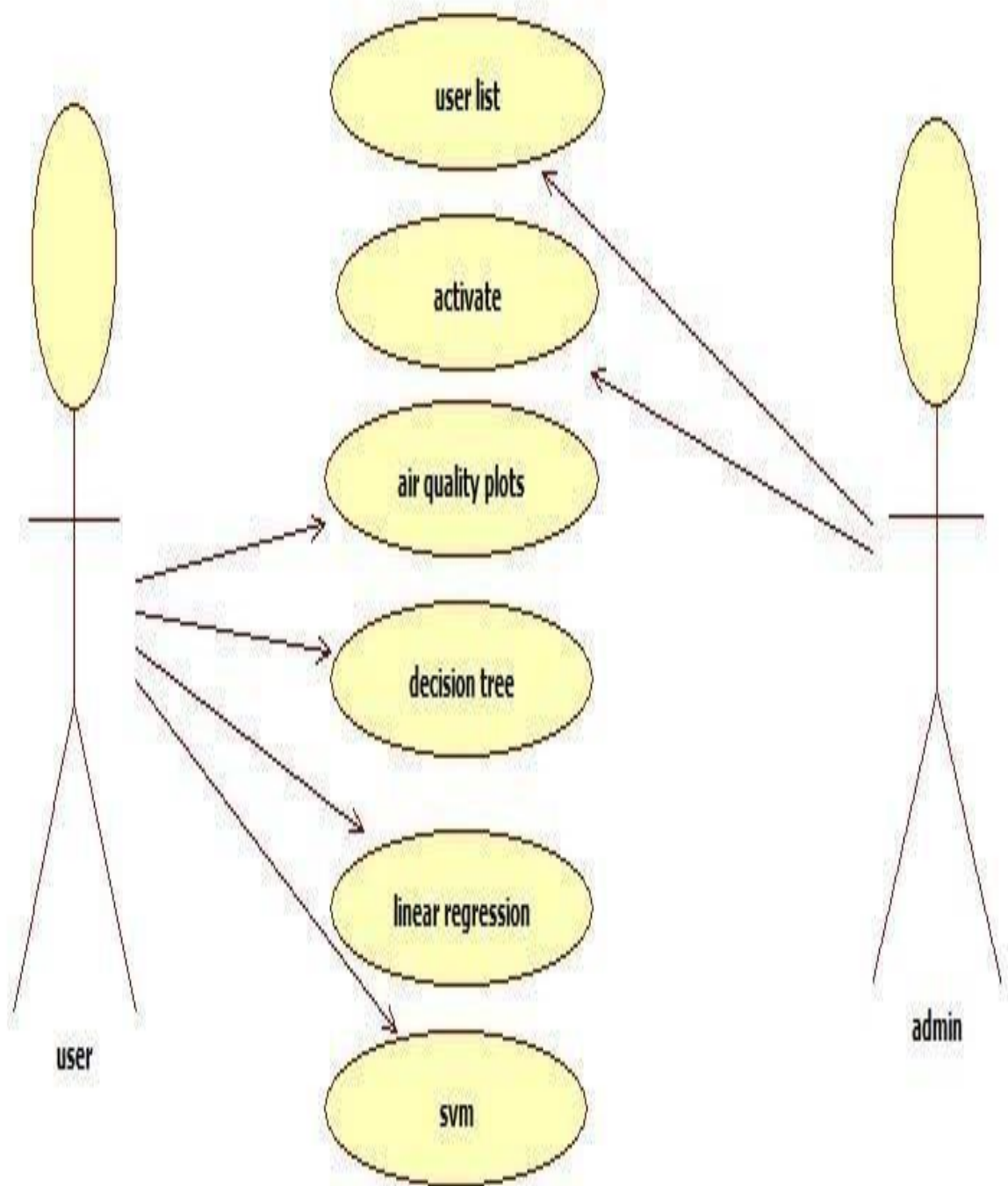


fig 4.2.1 Use Case Diagram

4.2.2 Sequence Diagram

A sequence diagram in Unified Modeling Language (UML) is a kind of interaction diagram that shows how processes operate with one another and in what order. It is a construct of a Message Sequence Chart. Sequence diagrams are sometimes called event diagrams, event scenarios, and timing diagrams.

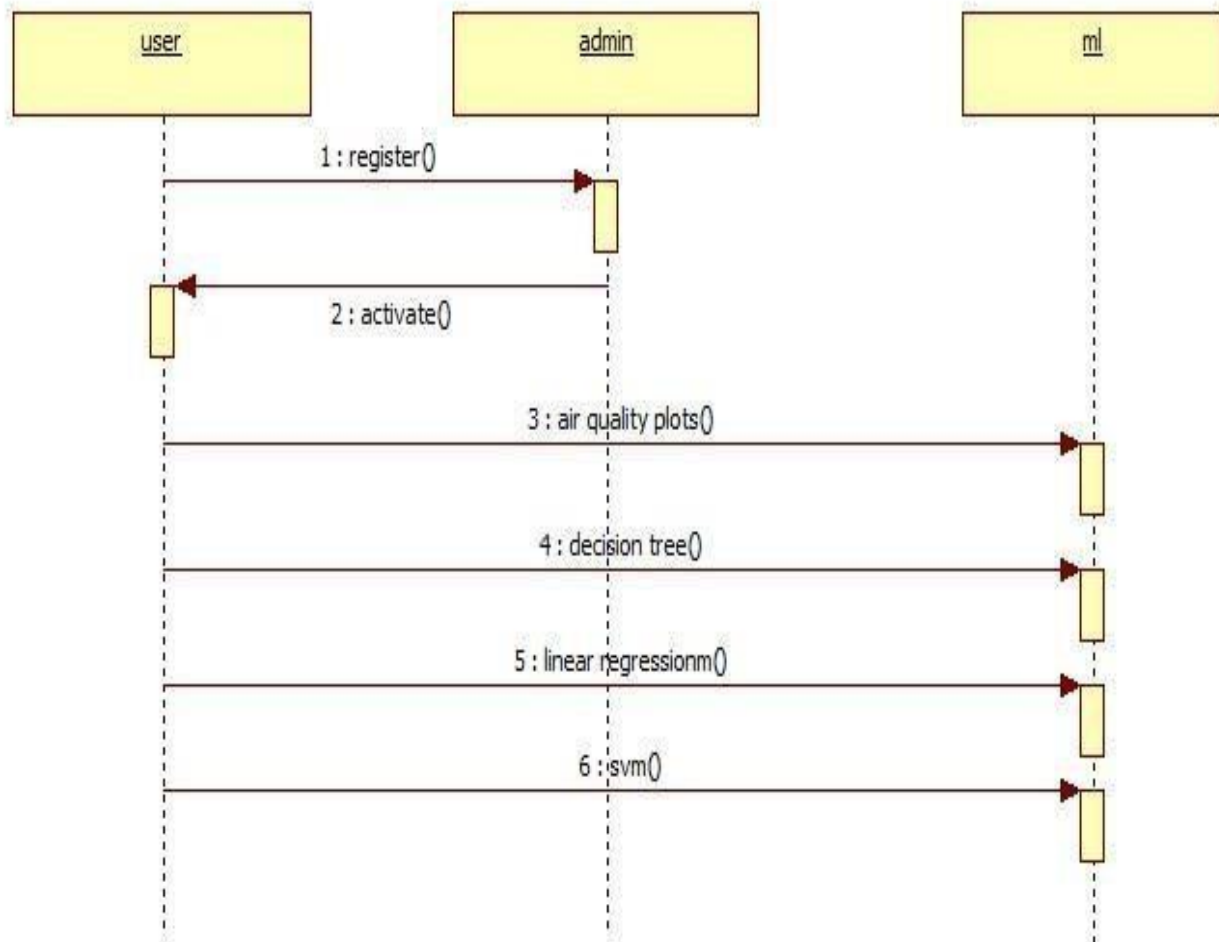
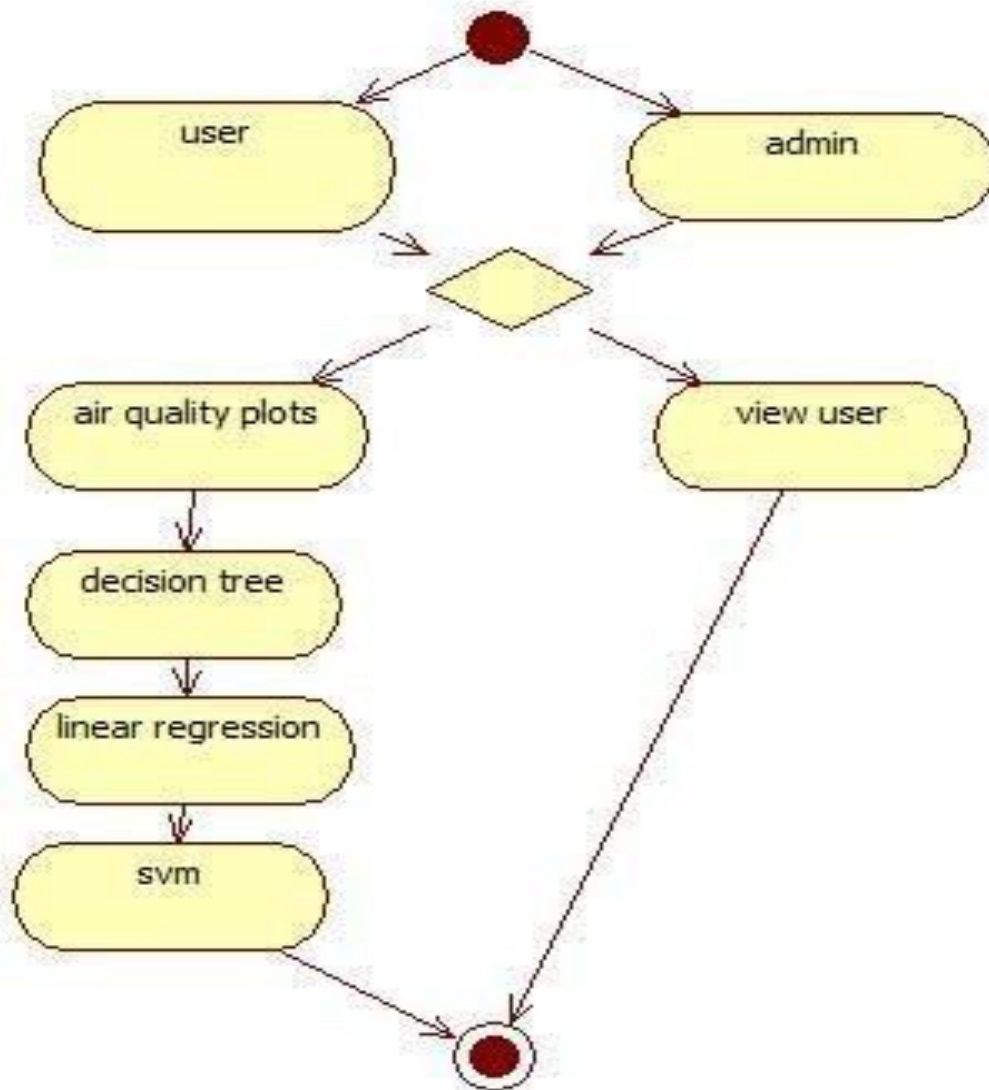


Fig 4.2.2 Sequence Diagram

4.2.3ActivityDiagram

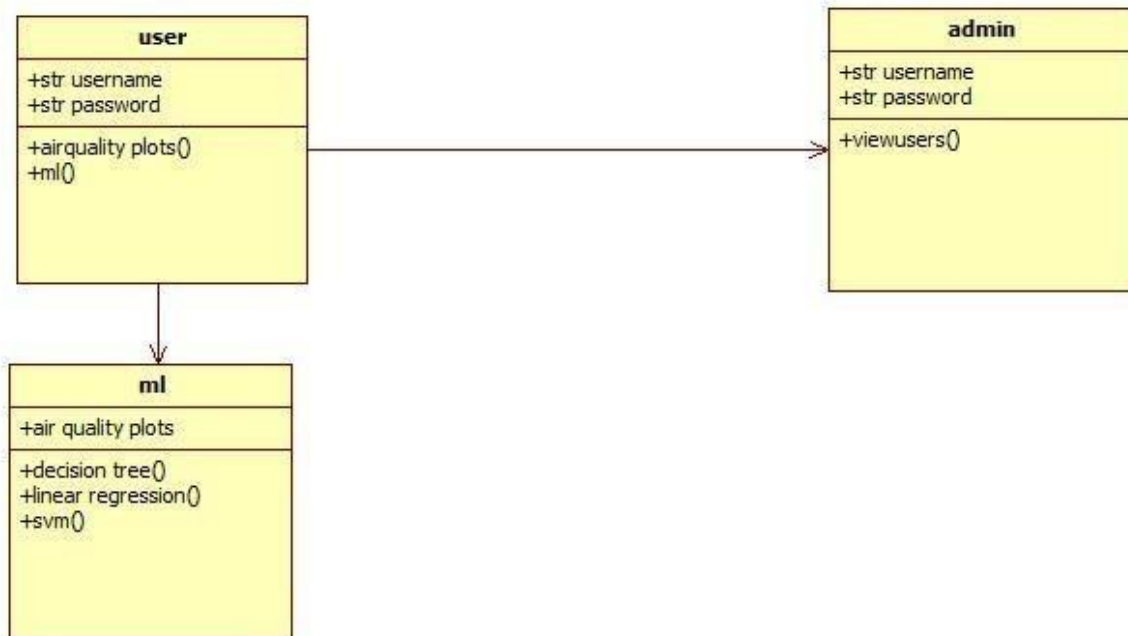
Activity diagrams are graphical representations of workflows of stepwise activities and actions with support for choice, iteration and concurrency. In the Unified Modeling Language, activity diagrams can be used to describe the business and operational step-by-step workflows of components in a system. An activity diagram shows the overall flow of control.



4.2.3 Activity Diagram

4.2.4 Class Diagram

In software engineering, a class diagram in the Unified Modeling Language (UML) is a type of static structure diagram that describes the structure of a system by showing the system's classes, their attributes, operations (or methods), and the relationships among the classes. It explains which class contains information.

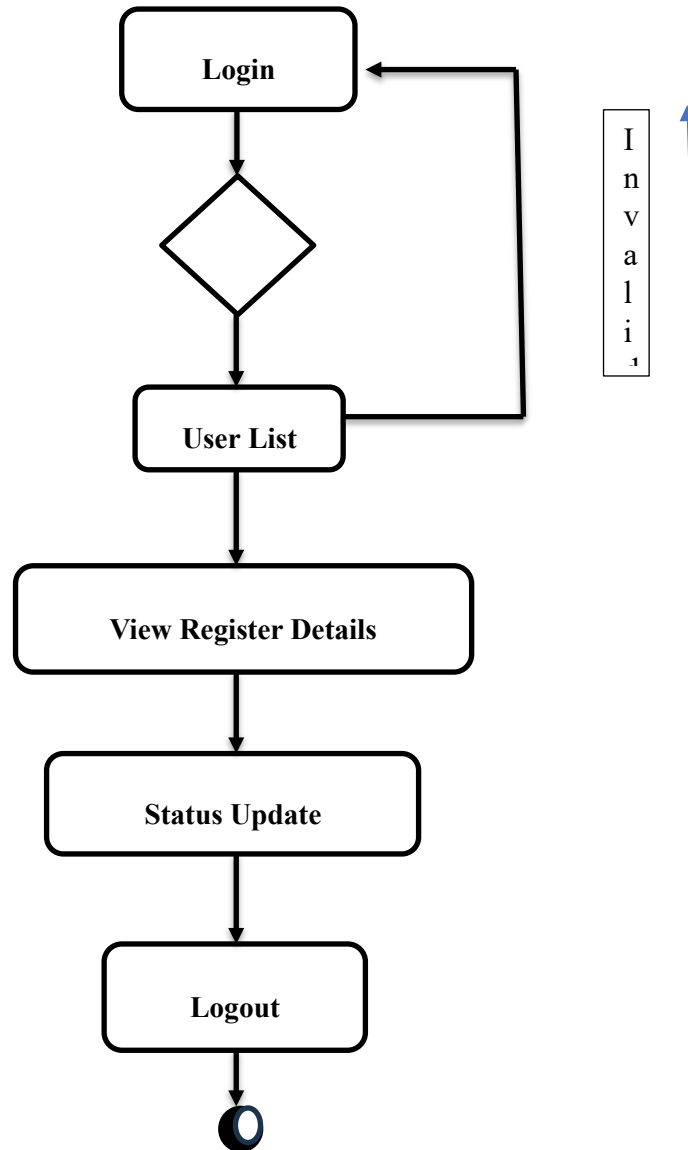


4.2.4 Class Diagram

Flow Chart Diagram:

1. The DFD is also called as bubble chart. It is a simple graphical formalism that can be used to represent a system in terms of input data to the system, various processing carried out on this data, and the output data is generated by this system.
2. The data flow diagram (DFD) is one of the most important modeling tools. It is used to model the system components. These components are the system process, the data used by the process, an external entity that interacts with the system and the information flows in the system.
3. DFD shows how the information moves through the system and how it is modified by a series of transformations. It is a graphical technique that depicts information flow and the transformations that are applied as data moves from input to output.

4. DFD is also known as bubble chart. A DFD may be used to represent a system at any level of abstraction. DFD may be partitioned into levels that represent increasing information flow and functional detail.



4.2.5 Flow Chat Diagram

SYSTEM IMPLEMENTATION

5.1 MODULES

MODULES DESCRIPTION:

User:

The User can register the first. While registering he required a valid user email and mobile for further communications. Once the user registers, then admin can activate the customer. Once the admin activates the customer then the customer can login into our system. After login he can add the data to predict the traffic prediction . After adding the data we can find the prediction of the algorithm. First we can find the svm algorithm and then we can find the random forest algorithm.

Admin:

Admin can login with his credentials. Once he logs in he can activate the users. The activated user only login in our applications. The admin can set the predictions of algorithms. Admin can predict random forest algorithms and also predict the support vector machine algorithm. The admin can add new data to the dataset.

svm:

Support Vector Machine” (SVM) is a supervised machine learning algorithm which can be used for both classification or regression challenges. However, it is mostly used in classification problems. In the

SVM algorithm, we plot each data item as a point in n-dimensional space (where n is number of features you have) with the value of each feature being the value of a particular coordinate. Then, we perform classification by finding the hyper-plane that differentiates the two classes very well

Decision Trees:

Versatile machine learning tools for classification and regression tasks. They represent decisions as a tree structure, guiding through feature-based choices to predict outcomes. Decision Trees are interpretable, prone to overfitting (addressed through pruning), and can be enhanced via ensemble methods like Random Forest and Gradient Boosting.

5.2 Requirement Specifications

5.2.1 Minimum Hardware Requirements

- Processor: Intel i5 or equivalent (2.0 GHz+)
- RAM: 4 GB minimum
- Camera: HD Webcam (720p or better)
- Storage: Minimum 1 GB free disk space

5.2.2 Software Requirements

- OS: Windows 10 / Linux / macOS
- Python: Version 3.8 or higher
- Libraries: OpenCV, DeepFace, NumPy, pandas
- Database: SQLite or MySQL
- IDE: Visual Studio Code / PyCharm

TECHNOLOGY DESCRIPTION

6.1 Introduction to Python

PYTHON

Overview of Python Programming Language:

Python is a versatile, high-level programming language known for its readability and simplicity. It's interpreted and supports interactive use, which means you can run commands directly in the interpreter without writing a complete script. One of Python's key design features is the use of indentation (instead of braces) to define blocks of code, making it easy to read and maintain.

Python supports various programming paradigms including object-oriented, procedural, functional, and imperative styles. It features dynamic typing and automatic memory management, making it beginner-friendly and powerful for advanced tasks as well.

The official Python implementation, CPython, is open-source and maintained by the Python Software Foundation, with a wide community contributing to its development. It runs on multiple platforms and comes with a robust standard library.

Programming Modes:

- **Interactive Mode:** You can directly type commands into the Python shell. For example:

```
>>> print("Hello, Python!")
```
- **Script Mode:** You can save your code in a `.py` file (e.g., `test.py`) and execute it as a complete script.

Python Syntax Basics:

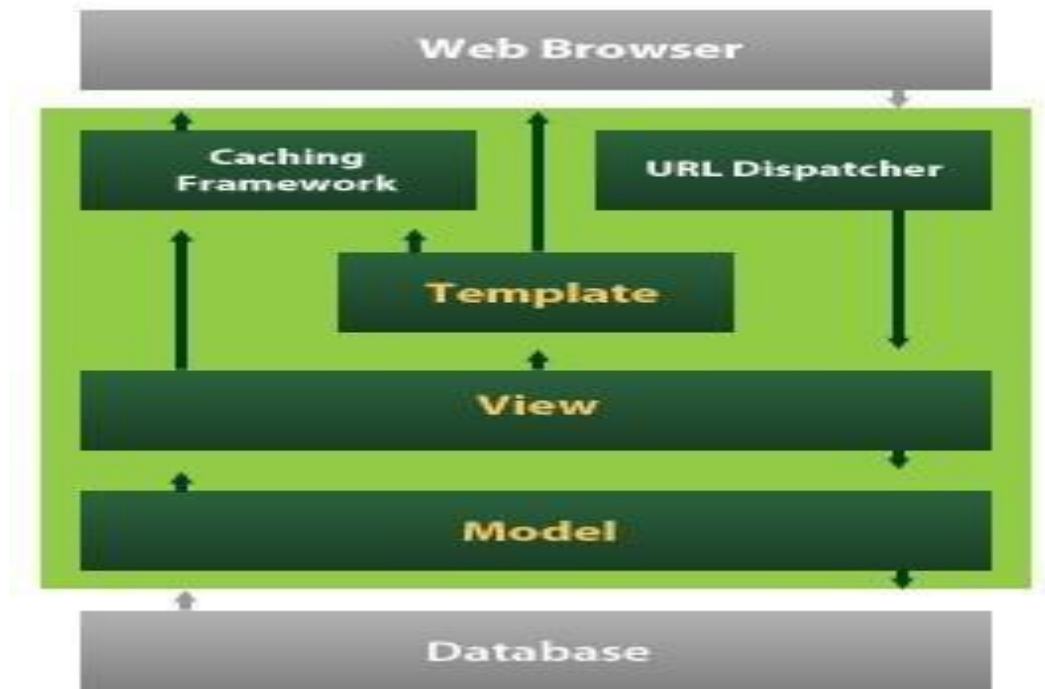
- **Identifiers:** Names used for variables, functions, etc., must start with a letter or underscore and can include numbers. They are case-sensitive.
- **Naming Conventions:**
 - Class names start with uppercase letters.
 - Private identifiers begin with an underscore (`_`), and strongly private ones with two underscores (`__`).
 - Special methods (like `__init__`) begin and end with double underscores.

Reserved Keywords: Python has reserved words such as `if`, `else`, `while`, `for`, `try`, `class`, and `def` that cannot be used as identifiers.

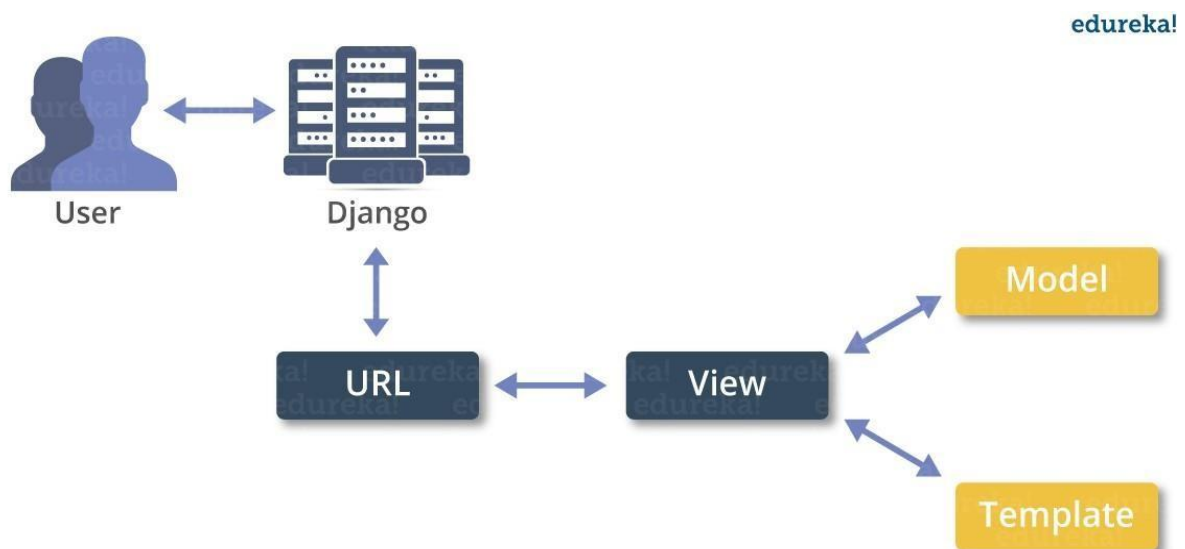
Indentation Rules: Python uses indentation to define code blocks. All lines within a block must be indented equally, otherwise, an error will occur. For instance:

DJANGO

Django is a robust Python web framework promoting fast, efficient app creation. It simplifies building dynamic, database-backed websites with minimal effort. Key principles include reusability, modularity, and avoiding code duplication. Entirely Python-based, it supports clean design across models, views, and settings. Open-source and developer-friendly, Django accelerates scalable web development.



Django also provides an optional administrative create,read,delete & update interface that is generated dynamically through introspection and configured via admin models



Absolutely! Here's your content formatted as a **clean, readable passage** — like you'd find in a textbook or tutorial article. You can use this directly in documentation, a blog, or even for study notes. Absolutely! Here's your content formatted as a **clean, readable passage** — like you'd find in a textbook or tutorial article. You can use this directly in documentation, a blog, or even for study notes.

Creating a Django Project from Scratch

To start building a Django project, open a terminal (on Linux or Mac) or Command Prompt (on Windows), navigate to your desired working directory, and run the following command:

```
django-admin startproject myproject
```

This creates a folder named `myproject` with the following structure:

```
myproject/
|  manage.py
└─myproject/
    |  __init__.py
    |  settings.py
    |  urls.py
    |  wsgi.py
```

Project Structure Overview

- **manage.py**: A command-line utility to interact with your Django project (e.g., running the server, database migrations).
 - **myproject/** (inner folder): The actual project package, which includes:
 - `__init__.py`: Makes the folder a Python package.
 - `settings.py`: Contains configuration settings like database, timezone, middleware, etc.
 - `urls.py`: Acts like a Table of Contents for your project, mapping URLs to views.
 - `wsgi.py`: Used for deploying the project to a WSGI server in production.
-

Setting Up the Project

Inside settings.py, a few key options must be configured:

- **DEBUG = True**
Enables detailed error messages and static file serving during development. Be sure to set it to False in production.
- **DATABASES**
Configure your project database. Example using SQLite:

```
DATABASES = {  
  
    'default': {  
  
        'ENGINE': 'django.db.backends.sqlite3',  
  
        'NAME': 'database.sql',  
  
    }  
}
```

Other supported databases include:

- MySQL: django.db.backends.mysql
- PostgreSQL: django.db.backends.postgresql
- Oracle: django.db.backends.oracle
- MongoDB (with third-party support): django_mongodb_engine

To run your project locally and check everything works:

```
python manage.py runserver
```

You'll see a message like:

Starting development server at <http://127.0.0.1:8000/>

Django Apps and Their Role

In Django, a **project** is composed of multiple **apps**, each designed to perform a specific function. For example, a contact form could be one app, and a blog could be another — both reusable in other projects.

To create an app inside your project:

```
python manage.py startapp myapp
```

This generates the following structure:

```
myapp/
```

```
| __init__.py
```

```
| admin.py
```

```
| models.py
```

```
| tests.py
```

```
| views.py
```

App File Roles

- `admin.py`: Customize Django Admin for your app.
 - `models.py`: Define your database models here.
 - `views.py`: Handle request/response logic.
 - `tests.py`: Write unit tests.
 - `__init__.py`: Marks the directory as a Python package.
-

Registering the App

To make your project aware of the new app, add it to `INSTALLED_APPS` in `settings.py`:

```
INSTALLED_APPS = [
```

```
...
```

```
'myapp',
```

```
]
```

Creating Forms in Django

To create a form, define a class inheriting from `forms.Form`.

Example: `myapp/forms.py`

```
from django import forms
```

```
class LoginForm(forms.Form):
```

```
    user = forms.CharField(max_length=100)
```

```
    password = forms.CharField(widget=forms.PasswordInput())
```

Widgets like `PasswordInput`, `DateInput`, or `CheckboxInput` allow better control of how the form fields render in HTML.

Handling CSRF and Sessions

Django provides built-in CSRF protection. Include `{% csrf_token %}` in your form templates to protect against CSRF attacks.

To enable sessions, update your `settings.py` file:

- Add to `MIDDLEWARE`:

```
'django.contrib.sessions.middleware.SessionMiddleware',
```

- Add to `INSTALLED_APPS`:

```
'django.contrib.sessions',
```

Login System Using Sessions

Here's a sample login view that stores the user's name in the session:

```
def login(request):
```

```
    username = 'not logged in'
```

```
    if request.method == 'POST':
```

```
MyLoginForm = LoginForm(request.POST)
```

```
if MyLoginForm.is_valid():
```

```
    username = MyLoginForm.cleaned_data['user']
```

```
    request.session['username'] = username
```

```
else:
```

```
    MyLoginForm = LoginForm()
```

```
    return render(request, 'loggedin.html', {"username": username})
```

Now create a view that only shows the login form if the user is not already logged in:

```
def formView(request):
```

```
    if request.session.has_key('username'):
```

```
        username = request.session['username']
```

```
        return render(request, 'loggedin.html', {"username": username})
```

```
    else:
```

```
        return render(request, 'login.html', {})
```

Updating URLs

Finally, map these views in your app's `urls.py`:

```
from django.conf.urls import url
```

```
from myapp import views
```

```
urlpatterns = [
```

```
    url(r'^connection/$', views.formView, name='loginform'),
```

```
url(r'^login/$', views.login, name='login'),
```

```
]
```

Now, when you visit `/connection`, you'll either see the login form or be redirected to the logged-in page based on session data.

Creating a Django Project from Scratch

To start building a Django project, open a terminal (on Linux or Mac) or Command Prompt (on Windows), navigate to your desired working directory, and run the following command:

```
django-admin startproject myproject
```

This creates a folder named `myproject` with the following structure:

```
myproject/
├── manage.py
└── myproject/
    ├── __init__.py
    ├── settings.py
    ├── urls.py
    └── wsgi.py
```

Project Structure Overview

- **manage.py**: A command-line utility to interact with your Django project (e.g., running the server, database migrations).
- **myproject/** (inner folder): The actual project package, which includes:
 - `__init__.py`: Makes the folder a Python package.
 - `settings.py`: Contains configuration settings like database, timezone, middleware, etc.
 - `urls.py`: Acts like a Table of Contents for your project, mapping URLs to views.
 - `wsgi.py`: Used for deploying the project to a WSGI server in production.

SAMPLE CODE

Algorithm.py

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder

def process_data(request):
    path = 'your/path/to/csv/file'
    data = pd.read_csv(path)

    # Preprocessing
    features = ['PM2.5', 'PM10', 'NO', 'NO2', 'NOx', 'NH3', 'CO',
'SO2', 'O3']
    for feature in features:
        data[feature].fillna(data[feature].mean(), inplace=True)

    le = LabelEncoder()
    data['bucket_encoded'] = le.fit_transform(data['AQI_Bucket'])

    X = data[features]
    y = data['bucket_encoded']

    X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2)

    model = DecisionTreeClassifier()
    model.fit(X_train, y_train)

    return model, le, X_train, X_test, y_train, y_test
```

```
from django.shortcuts import render
from django.contrib import messages
from ..forms import UserRegistrationForm
from ..models import UserRegistrationModel

def UserRegisterActions(request):
    if request.method == 'POST':
        form = UserRegistrationForm(request.POST)
        if form.is_valid():
            form.save()
            messages.success(request, 'You have been successfully
registered')
            return render(request, 'UserRegistrations.html', {'form':
UserRegistrationForm()})
        else:
            messages.error(request, 'Invalid data or user already
exists')
    else:
        form = UserRegistrationForm()
        return render(request, 'UserRegistrations.html', {'form': form})

def UserLoginCheck(request):
    if request.method == "POST":
        loginid = request.POST.get('loginid')
        pswd = request.POST.get('pswd')
        try:
            user = UserRegistrationModel.objects.get(loginid=loginid,
password=pswd)
            if user.status == "activated":
                request.session['id'] = user.id
                request.session['loggeduser'] = user.name
                request.session['loginid'] = loginid
                return render(request, 'users/UserHomePage.html')
            else:
                messages.error(request, 'Your account is not
activated')
        except UserRegistrationModel.DoesNotExist:
            messages.error(request, 'Invalid login credentials')
    return render(request, 'UserLogin.html')
```

```
from django.shortcuts import render
from ..algorithms.Algorithm import process_data
from sklearn.metrics import mean_squared_error
import pandas as pd

def predict_aqi_bucket(request):
    if request.method == 'POST':
        input_data = pd.DataFrame({
            'PM2.5': [float(request.POST['pm25'])],
            'PM10': [float(request.POST['pm10'])],
            'NO': [float(request.POST['no'])],
            'NO2': [float(request.POST['no2'])],
            'NOx': [float(request.POST['nox'])],
            'NH3': [float(request.POST['nh3'])],
            'CO': [float(request.POST['co'])],
            'SO2': [float(request.POST['so2'])],
            'O3': [float(request.POST['o3'])],
        })

        model, le, X_train, X_test, y_train, y_test =
process_data(request)
        prediction = model.predict(input_data.values)
        predicted_label =
le.inverse_transform(prediction.astype(int))[0]

        mse_train = mean_squared_error(y_train,
model.predict(X_train))
        mse_test = mean_squared_error(y_test, model.predict(X_test))

        return render(request, 'users/result.html', {
            'predicted_bucket': predicted_label,
            'mse_train': mse_train,
            'mse_test': mse_test,
        })

    return render(request, 'users/prediction_form.html')
```

Visualizations

```
from django.shortcuts import render
import pandas as pd
import plotly.express as px
from django.conf import settings

def air_quality_pair_plots(request):
    path = settings.MEDIA_ROOT + "/city_day.csv"
    data = pd.read_csv(path)
    columns = ['PM2.5', 'PM10', 'NO', 'NO2', 'NOx', 'NH3', 'CO', 'SO2', 'O3']
    data = data[columns].dropna()
    fig = px.scatter_matrix(data, dimensions=columns)
    plot_html = fig.to_html(full_html=False, include_plotlyjs='cdn')
    return render(request, 'users/pair_plots.html', {'plot_html': plot_html})
```

Decision Tree

```
from django.shortcuts import render
from django.conf import settings
import pandas as pd

def decision_tree(request):
    path = settings.MEDIA_ROOT + "/city_day.csv"
    df = pd.read_csv(path)

    features = ['PM2.5', 'PM10', 'NO', 'NO2', 'NOx', 'NH3', 'CO', 'SO2', 'O3']
    for col in features:
        df[col].fillna(df[col].mean(), inplace=True)

    # Training logic here...
    return render(request, 'users/decision_tree_summary.html')
```

URLs.py

```
from django.shortcuts import render
from django.conf import settings
import pandas as pd

def decision_tree(request):
    path = settings.MEDIA_ROOT + "/city_day.csv"
    df = pd.read_csv(path)

    features = ['PM2.5', 'PM10', 'NO', 'NO2', 'NOx', 'NH3', 'CO', 'SO2', 'O3']
    for col in features:
        df[col].fillna(df[col].mean(), inplace=True)

    # Training logic here...
    return render(request, 'users/decision_tree_summary.html')
```

TESTING

8.1. SYSTEM TEST

System Testing and Types of Software Testing – Summary

Software testing is the process of verifying that a software system functions correctly, meets user requirements, and operates without failure. It aims to identify faults at various levels of development to ensure a high-quality product. Several testing types serve different purposes, from validating individual functions to confirming the entire system works as expected.

Types of Software Testing

1. **Unit_Testing**

This involves testing individual units or components of a program to ensure each part works as intended. It focuses on internal logic, control flow, and data handling. Unit tests are typically done after development of each module and require understanding of the code structure.

2. **Integration_Testing**

Integration testing verifies the interaction between combined units. Although individual modules may work fine in isolation, their integration can cause issues. This test ensures combined components work together correctly and consistently.

3. **Functional_Testing**

Functional testing assesses whether the software behaves according to its specifications. It checks valid/invalid input, expected outputs, and that all features are operational. It also includes interface testing and ensures each business function flows correctly.

4. **System_Testing**

System testing validates the behavior of the entire software system. It confirms the integrated system meets both technical and business requirements. This test mimics real-world usage and is based on defined processes and workflows.

5. **White_Box_Testing**

This type involves testing with full knowledge of the internal code structure. Testers analyze logic paths, loops, and conditions to ensure thorough coverage. It is used where deep inspection is required.

6. **Black_Box_Testing**

Black box testing is done without access to the source code. The tester focuses on input/output and external behavior. This method relies on the software's documentation and specifications.

7. **Acceptance_Testing**

Performed by end users, this phase checks if the application meets real-world requirements. It ensures user needs are fulfilled before deployment. All functionalities are validated from the user's perspective.

Test Strategy & Objectives

- All field inputs must operate accurately.
- Links and navigations should work as expected.
- Response times, messages, and page loads must not delay or break.
- Inputs should be in valid formats, and duplicates must be prevented.

Testing Results Summary

- All planned test cases across different testing stages (unit, integration, system, and acceptance) passed successfully.
- No critical defects were identified.
- The software demonstrated stable and expected behavior throughout testing phases.

8.2 Test cases

Sample Test Cases

S.no	Test Case	Excepted	Result	Remarks
1	User Register	If User registration successfully.	Pass	If already user email exists then it fails.
2	User Login	If the Username and password is correct then it will be a valid page.	Pass	Un Register Users will not log in.
3	Admin Add	A new record will be added to our dataset.	Pass	According to Indiameteorological repository the data must be float fields otherwise it fails.
4	user adddata	user will add the data to find out location prediction based on details	Pass	we couldn't get the required data to predict..
5	randomforest	will get the random forest accuracy results.	Pass	won't get accuracy
6	svm	will get the svm result	Pass	won't get accuracy

7	Linear regression	Will get linear regression accuracy	pass	won't get accuracy
8	Decision tree	Will get decision tree accuracy	Pass	won't get accuracy
9	Admin login	Admin can login with his login credential. If success he get his home page	Pass	Invalid login details will not allowed here
10	Admin can activate the register users	Admin can activate the register user id	Pass	If the user did not find it then it won't login.



SCREEN SHOTS

9.SCREENSHOTS

Home Page

Air Quality Prediction based on Machine Learning

[HOME](#) [USER](#) [ADMIN](#) [REGISTER](#)

fig7.1 This fig shows the first screen of our project

Register form

The screenshot displays the 'User Register Form' on the 'HOME' page of the 'Air Quality Prediction based on Machine Learning' application. The form is centered and contains the following fields: User Name, Login ID, Password, Mobile, email, Locality, Address (a larger text area), City, and State. A blue 'Register' button is positioned at the bottom of the form. The navigation bar at the top includes links for HOME, USER, ADMIN, and REGISTER, with HOME being the active page.

Air Quality Prediction based on Machine Learning

[HOME](#) [USER](#) [ADMIN](#) [REGISTER](#)

User Register Form

User Name

Login ID

Password

Mobile

email

Locality

Address

City

State

[Register](#)

Admin login

Air Quality Prediction based on Machine Learning

HOMEUSERADMINREGISTER

Admin Login Form

Enter Login Id

Enter password

LoginReset

Admin home

Air Quality Prediction based on Machine Learning

HOMEUSERADMINREGISTER

Admin Login Form

Enter Login Id

Enter password

LoginReset

User details

[HOME](#) [USER LIST](#) [LOGOUT](#)

View RegisterUser Details

S.No	Name	Login ID	Mobile	Email	Locality	Status	Activate
1	shazz	shazz	7093628022	shaistafatima860@gmail.com	hyd	activated	Activated

user login page

Air Quality Prediction based on Machine Learning

[HOME](#) [USER](#) [ADMIN](#)

User Login Form

admin

.....

Login

Reset

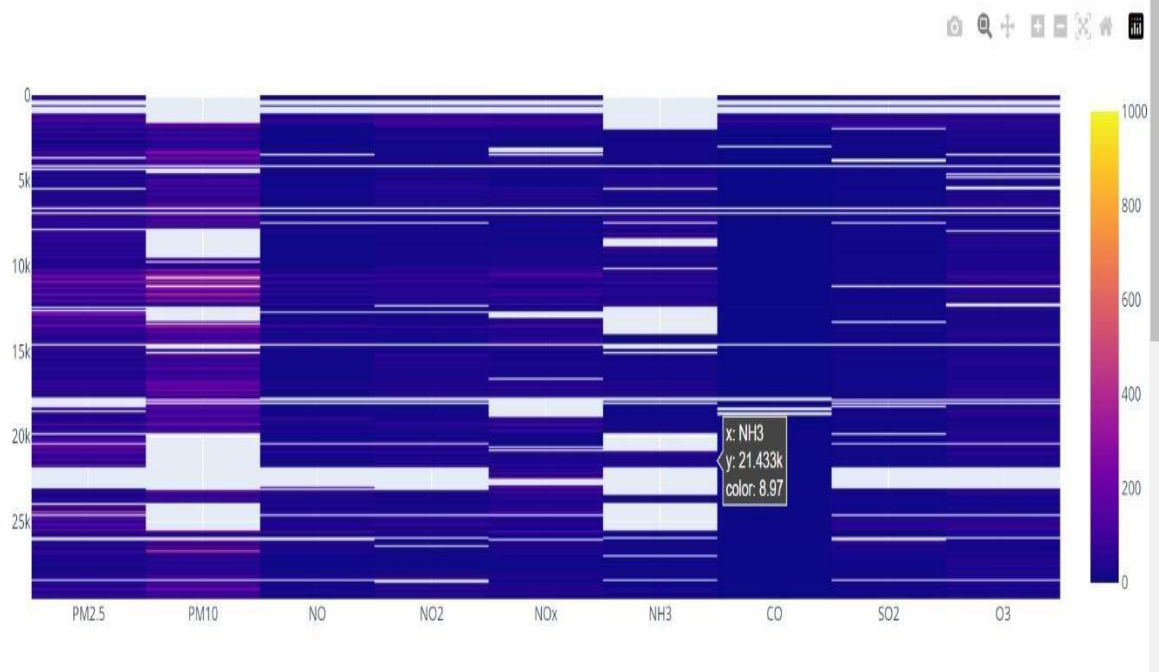
User home page

HOME AIR_QUALITY_PAIR_PLOTS DECISION_TREE LINEAR REGRESSION SVM PREDICT LOGOUT

Welcome To User HomePage.....

Air quality plots:

Air Quality Pair Plots



Metrics for support vector machine

Mean Absolute Error (MAE)

1.1864548189252768

Mean Squared Error (MSE)

2.6270295422198853

Root Mean Squared Error
(RMSE)

1.6208113839123555

Decision tree

Metrics for decision tree

Mean Absolute Error (MAE)

0.6515095988951972

Mean Squared Error (MSE)

1.9134658938800835

Root Mean Squared Error
(RMSE)

1.383280844181717

Metrics for linear regressor

Mean Absolute Error (MAE)

1.50706613645483

Mean Squared Error (MSE)

3.1870272822999635

Root Mean Squared Error
(RMSE)

1.7852247147908196

PREDICTION FORM

Predict AQI Bucket

PM2.5:

PM10:

NO:

NO2:

NOx:

NH3:

CO:

SO2:

O3:

Predict AQI Bucket

RESULT

Prediction Result

Prediction Result:

Predicted AQI Bucket:

Air quality is Moderate

MSE on Training Data:

0.031154757873349138

MSE on Test Data:

0.03690744920993228

[Back to Prediction Form](#)

CONCLUSION

Conclusion

This project focuses on developing a reliable and efficient system for predicting pollution levels and assessing associated air quality risks. By utilizing a carefully selected set of features and applying machine learning algorithms, the model aims to achieve high prediction accuracy. The project leverages key environmental factors to evaluate air quality levels and assess potential risks. With the use of modern data-driven techniques, this system supports proactive environmental management and helps raise awareness about pollution exposure.

Future Enhancements

To further improve the performance and applicability of the air quality prediction system, several enhancements are proposed:

- **Advanced Feature Selection:** Use of more refined techniques to choose the most impactful variables.
- **Improved Algorithms:** Integration of cutting-edge models such as gradient boosting, deep learning, or hybrid ensembles for better precision.
- **Temporal & Spatial Analysis:** Incorporation of time-based trends and location-based variations to enhance prediction relevance.
- **Real-Time Monitoring:** Implementation of live data streaming and continuous feedback loops for instant updates.
- **Data Handling Techniques:** Use of data imputation and augmentation to deal with missing or inconsistent data.
- **Model Interpretability:** Emphasis on explainable AI for transparency in how predictions are made.
- **External Data Integration:** Merging external sources like weather, traffic, and satellite data for a more holistic view.
- **User Interface:** Developing intuitive dashboards or mobile applications to make insights accessible to non-technical users.
- **Benchmarking & Validation:** Comparing model performance with existing systems to ensure reliability and generalizability.

These enhancements would not only boost the system's accuracy but also increase its adaptability, making it more effective for real-world applications.

BIBLIOGRAPHY

Text Books

1. **Aurélien Géron**, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* – A practical guide to machine learning techniques using popular Python libraries.
2. **Christopher Bishop**, *Pattern Recognition and Machine Learning* – A foundational text offering in-depth knowledge on probabilistic models and pattern recognition.
3. *Air Pollution Modeling and its Application* – A series edited by **D.G. Steyn** and others, exploring various methodologies for air pollution analysis and simulation.

References

1. Ishan Verma, Rahul Ahuja, Hardik Meisheri, and Lipika Dey, “*Air Pollutant Severity Reduction using Bidirectional LSTM Network*,” IEEE/WIC/ACM International Conference on Web Intelligence, 2018.
2. Chao Zhang et al., “*Hybrid Measurement of Air Quality as a Mobile Service: An Image Based Approach*,” IEEE International Conference on Web Services (ICWS), 2017.
3. Raijin Yang, Feng Yan, and Nan Zhao, “*Urban Air Quality Based on Bayesian Network*,” IEEE International Conference on Communication Software and Networks (ICCSN), 2017.
4. Temesegan Walelign Ayele and Rutvik Mehta, “*Air Pollution Monitoring and Prediction using IoT*,” International Conference on Inventive Communication and Computational Technologies (ICICCT), 2018.
5. Nadjat Djebbri and Mounira Rouainia, “*Artificial Neural Networks Based Air Pollution Monitoring in Industrial Sites*,” International Conference on Engineering and Technology (ICET), 2017.
6. Dinesh Kumar, “*Evolving Differential Evolution Method with Random Forest for Prediction of Air Pollution*,” Procedia Computer Science, vol. 132, 2018.
7. Ningbo Jiang and Matthew L. Riley, “*Exploring the Utility of the Random Forest Method for Forecasting Ozone Pollution in Sydney*,” Journal of Environment Protection and Sustainable Development, 2015.
8. Vladimir Svetnik et al., “*Random Forest: A Classification and Regression Tool for Compound Classification and QSAR Modelling*,” Journal of Chemical Information and Computer Sciences, vol. 43(6), 2003.
9. GáŠhard Biau, “*Analysis of a Random Forest Model*,” Journal of Machine Learning Research, vol. 13, 2012.
10. Gérard Biau and Erwan Scornet, “*A Random Forest Guided Tour*,” Test, vol. 25(2), 2016.
11. Rosina Grimm et al., “*Digital Soil Mapping using Random Forests Analysis*,” Geoderma, vol. 146(1–2), 2008.